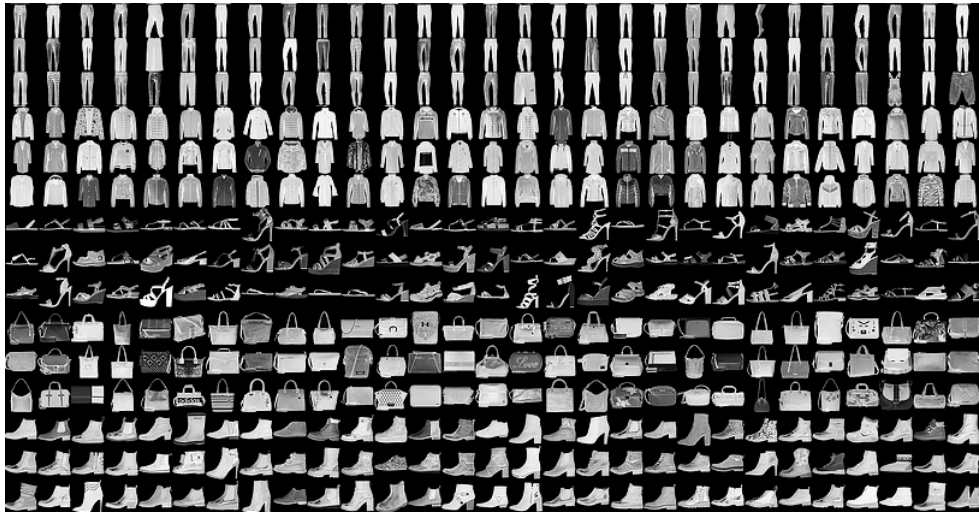


Deep Learning (Homework 1)

Due date : 2024/4/7 23:55:00 (Hard Deadline)

1 Feedforward Neural Network (60%)

You are given a dataset sampled from the [Fashion MNIST](#) dataset. This dataset contains 10 classes. In this exercise, you need to implement a [feedforward neural network \(FNN\)](#) model by yourself to recognize images and use the backpropagation algorithm to update the parameters.



Dataset Description:

- [Training set](#) contained 60,000 images. [Test set](#) contained 10,000 images.
- The images were 28 by 28 in size and were [flattened in row-major order into the shape of 784](#). The labels are integers that indicate the corresponding class. Details of these classes are provided in the above table.
- This dataset is given in the form of NumPy array files (.npz) named “[train_x.npz](#)”, “[train_y.npz](#)”, “[test_x.npz](#)” and “[test_y.npz](#)”, where “[x](#)” indicates input images and “[y](#)” indicates the corresponding labels.

Please follow the steps below to implement your program:

- Understand how the “forward pass” and “backward pass” in FNN work in accordance with the [backpropagation](#) algorithm. You can refer to this video.
- Both the training and test images need to be normalized ([divided by 255](#)).
- Use the [cross entropy](#) error function $J(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_k(\mathbf{x}_n, \mathbf{w})$ as the objective function where t_{nk} is the target value, N is the number of samples in a batch and $y_k(\mathbf{x}_n, \mathbf{w}_n)$ is the FNN output.

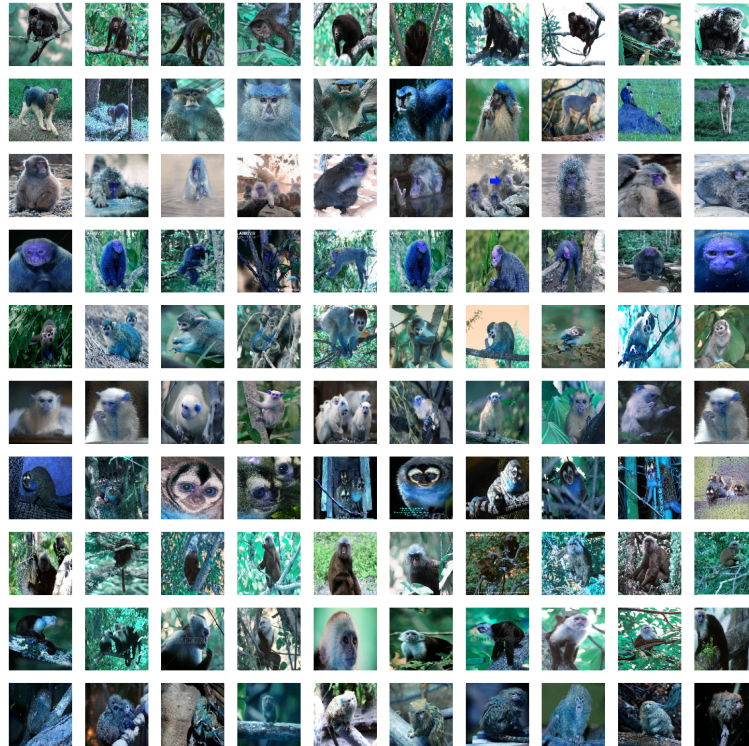
1. Design an FNN model architecture and use the file of the initial weights and biases “weights.npy”. Run the **backpropagation** algorithm and use the **mini-batch SGD** (stochastic gradient descent) $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla J(\mathbf{w}^{(\tau)})$ to optimize the parameters (the weights and biases), where η is the learning rate. **You should implement the FNN training under the following settings:**
 - number of layers: 3
 - number of neurons in each layer (in order): 2048, 512, 10
 - activation function for each layer (in order): relu, relu, softmax
 - number of training epochs: 30
 - learning rate: 0.01
 - batch size: 200
 - **important note:** For 1(a), **DO NOT RESHUFFLE THE DATA**. We had already shuffled the data for you. Reshuffling will make your result differ from our ground-truth result, and any difference will result in the reduction of your points. On the same note, when splitting the samples into batches, split them in the given sample order.
 - (a) **Plot** the learning curves of $J(\mathbf{w})$ and the accuracy of classification for every 25 iterations, with training data as well as test data, also, **show** the final loss and accuracy values. (20%)
 - (b) **Repeat** 1(a) by considering **zero initialization** for the model weights. And **make some discussion**. (8%)
2. Based on the model in 1, please **implement the dropout layers** and apply them after the first two hidden layers, i.e. the layers with 2048 and 512 neurons. The dropout rate should be set as 0.2 for both layers. Note that the dropout operation should only be applied in the training phase and should be disabled in the test phase.
 - (a) **Train** the model by using the same settings in 1 and **repeat** 1(a). (8%)
 - (b) Based on the experimental results, how do the dropout layers affect the model performance and why? Please **make some discussion**. (8%)
3. Based on the model in 1, please implement mini-batch SGD (stochastic gradient descent). In this problem, we need to reshuffle the data in every batch. Note that the other settings remain the same. Please set the random seed as **42**, and please use **random** library that we have imported.
 - (a) **Plot** the learning curves of $J(\mathbf{w})$ and the classification accuracy for every 25 iterations. Please **show** the final values of loss and accuracy. (8%)
 - (b) Based on the experimental results, how the process of reshuffling images affect the model performance and why? Please **make some discussion**. (8%)

Note:

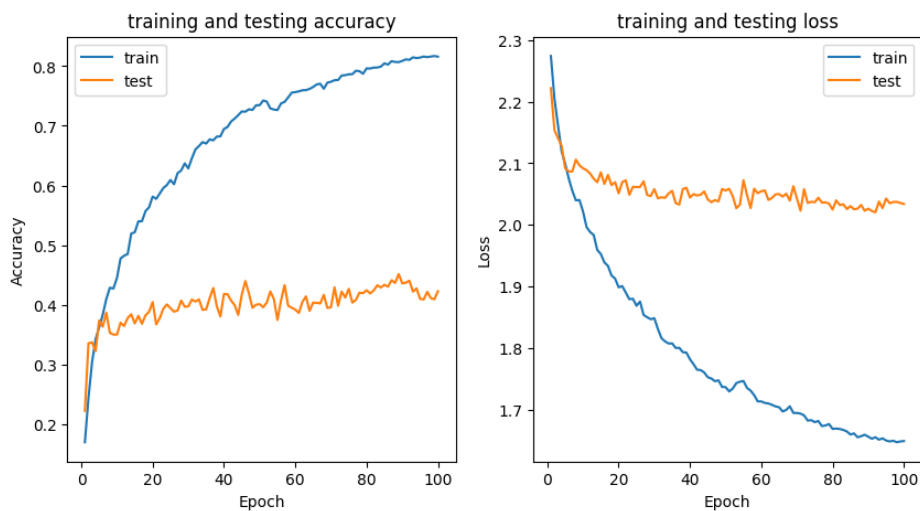
- When coding in Python, be careful to assign the value to a variable (**mutable vs immutable object**). Double-check the dimensions of your matrices.
- Normally, when training a deep neural network, you should shuffle the data for each epoch, but **for convenience of grading we restrict the data order in 1(a)**.

2 Convolutional Neural Network (40%)

In this exercise, you will construct a convolutional neural network (CNN) for image recognition by using **10 Monkey Species**. The training dataset consists of 1098 images and the testing dataset consists of 272 images. The dataset total has 10 categories.

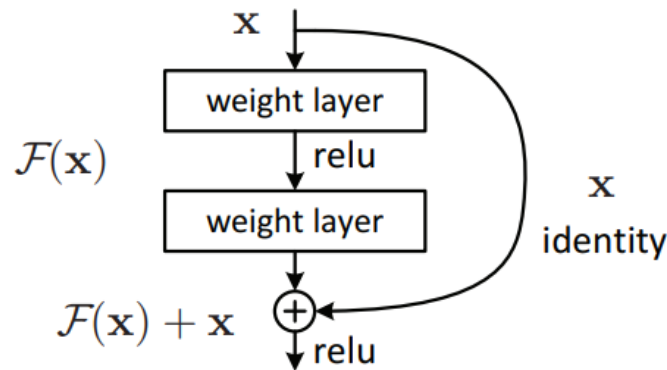


1. Please implement a CNN for image recognition by using **10 Monkey Species**, then **plot** the **learning curve** and the **accuracy rate** of training and test data. (20%)



NOTE: Figure above shows an example. The result might be different.

2. To deal with a real-world problem, we may stack some additional layers in the Deep Neural Network which results in improved accuracy and performance. However, it has been found that there is a maximum threshold for depth with the traditional convolutional neural network model. The problem of training a very deep network has been alleviated with the introduction of **ResNet** or residual network.



- (a) Construct a **ResNet** with residual blocks for image recognition and **plot** the [learning curve](#), [accuracy rate](#), try to stack more blocks as you can (ResNet-18 is recommended), you can refer to the [paper](#) for implementation. (15%)
- (b) Remove the identity mapping and repeat (a), then make some discussion on the results of (a) and (b). Please **describe** what you found. (5%)

NOTE: Please implement the model by yourself, directly loading the pre-trained model from Pytorch is not allowed.

3 Rule

- In your submission, you need to submit two files. And only the following file format is accepted:
 - **hw1_<StudentID>.ipynb** file which needs to contain all the results, codes and reports for each exercise (e.g. **hw1_0123456.ipynb**).
- Implementation will be graded by
 - Completeness
 - Algorithm correctness
 - Description of model design
 - Discussion and analysis
- Only [Python](#) implementation is acceptable.
- For [problem 1](#), any tools with [automatic differentiation](#) are [forbidden](#), such as [Tensorflow](#), [PyTorch](#), [Keras](#), etc. You should implement the backpropagation algorithm [by yourself](#).
- For [problem 2](#), you should use [PyTorch](#) to implement the model, [other deep learning APIs are forbidden](#).
- [DO NOT PLAGIARIZE](#). (We will check the program similarity score.)